

2751. Robot Collisions

Solved 

Hard

 Topics

 Companies

 Hint

There are n **1-indexed** robots, each having a position on a line, health, and movement direction.

You are given **0-indexed** integer arrays `positions`, `healths`, and a string `directions` (`directions[i]` is either 'L' for **left** or 'R' for **right**). All integers in `positions` are **unique**.

All robots start moving on the line **simultaneously** at the **same speed** in their given directions. If two robots ever share the same position while moving, they will **collide**.

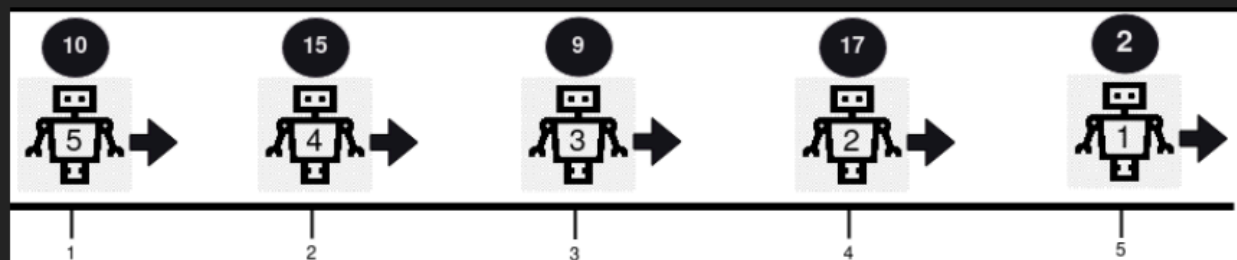
If two robots collide, the robot with **lower health** is **removed** from the line, and the health of the other robot **decreases by one**. The surviving robot continues in the **same** direction it was going. If both robots have the **same** health, they are both removed from the line.

Your task is to determine the **health** of the robots that survive the collisions, in the same **order** that the robots were given, i.e. final health of robot 1 (if survived), final health of robot 2 (if survived), and so on. If there are no survivors, return an empty array.

Return an array containing the health of the remaining robots (in the order they were given in the input), after no further collisions can occur.

Note: The positions may be unsorted.

Example 1:



Input: `positions = [5,4,3,2,1]`, `healths = [2,17,9,15,10]`, `directions = "RRRRR"`

Output: `[2,17,9,15,10]`

Explanation: No collision occurs in this example, since all robots are moving in the same direction. So, the health of the robots in order from the first robot is returned, `[2, 17, 9, 15, 10]`.

Example 2:



Input: `positions = [3,5,2,6]`, `healths = [10,10,15,12]`, `directions = "RLRL"`

Output: `[14]`

Explanation: There are 2 collisions in this example. Firstly, robot 1 and robot 2 will collide, and since both have the same health, they will be removed from the line. Next, robot 3 and robot 4 will collide and since robot 4's health is smaller, it gets removed, and robot 3's health becomes $15 - 1 = 14$. Only robot 3 remains, so we return `[14]`.

Example 3:



Input: `positions = [1,2,5,6]`, `healths = [10,10,11,11]`, `directions = "RLRL"`

Output: `[]`

Explanation: Robot 1 and robot 2 will collide and since both have the same health, they are both removed. Robot 3 and 4 will collide and since both have the same health, they are both removed. So, we return an empty array, `[]`.

Constraints:

- `1 <= positions.length == healths.length == directions.length == n <= 105`
- `1 <= positions[i], healths[i] <= 109`
- `directions[i] == 'L' or directions[i] == 'R'`
- All values in `positions` are distinct

Python:

```
class Solution:
    def survivedRobotsHealths(self, positions, healths, directions):

        n = len(positions)
        order = sorted(range(n), key=lambda i: positions[i])

        h = healths[:]
        alive = [True]*n
        stack = []

        for idx in order:

            if directions[idx] == 'R':
                stack.append(idx)

            else:
                while stack:

                    top = stack[-1]

                    if h[top] < h[idx]:
                        alive[top] = False
                        stack.pop()
                        h[idx] -= 1

                    elif h[top] > h[idx]:
                        alive[idx] = False
                        h[top] -= 1
                        break

                    else:
                        alive[top] = False
                        alive[idx] = False
                        stack.pop()
                        break

        return [h[i] for i in range(n) if alive[i]]
```

JavaScript:

```
var survivedRobotsHealths = function(pos, h, d) {

    const n = pos.length;
    const order = [...Array(n).keys()].sort((a,b)=>pos[a]-pos[b]);
```

```

const alive = Array(n).fill(true);
const stack = [];

for(const idx of order){

    if(d[idx]==='R'){
        stack.push(idx);
    } else {

        while(stack.length){

            let top = stack[stack.length-1];

            if(h[top] < h[idx]){
                alive[top]=false;
                stack.pop();
                h[idx]--;
            }
            else if(h[top] > h[idx]){
                alive[idx]=false;
                h[top]--;
                break;
            }
            else{
                alive[top]=false;
                alive[idx]=false;
                stack.pop();
                break;
            }
        }
    }
}

const res=[];
for(let i=0;i<n;i++)
    if(alive[i]) res.push(h[i]);

return res;
};

```

Java:

```

class Solution {
    public List<Integer> survivedRobotsHealths(int[] pos, int[] h, String d) {

```

```

int n = pos.length;
Integer[] order = new Integer[n];
for(int i=0;i<n;i++) order[i]=i;

Arrays.sort(order,(a,b)->pos[a]-pos[b]);

boolean[] alive = new boolean[n];
Arrays.fill(alive,true);

Deque<Integer> st = new ArrayDeque<>();

for(int idx:order){

    if(d.charAt(idx)=='R') st.push(idx);

    else{
        while(!st.isEmpty()){

            int top = st.peek();

            if(h[top] < h[idx]){
                alive[top]=false;
                st.pop();
                h[idx]--;
            }
            else if(h[top] > h[idx]){
                alive[idx]=false;
                h[top]--;
                break;
            }
            else{
                alive[top]=false;
                alive[idx]=false;
                st.pop();
                break;
            }
        }
    }
}

List<Integer> res=new ArrayList<>();
for(int i=0;i<n;i++)
    if(alive[i]) res.add(h[i]);

```

```
        return res;
    }
}
```